Examen FIS - Raspunsuri 2025

Varianta 1 examen

1. a. Curba defectelor poate fi facuta sa tinda in timp spre zero daca cerintele sunt bine intelese de la inceput si daca se aplica sistematic metode de testare eficiente
   `FALS. Curba defectelor poate fi facuta sa tinda spre 0 doar intr-un caz ideal, lucru inexistent de altfel. In realitate, curba defectelor se indeparteaza de 0 pe masura ce se aplica schimbari sistemului SW (inclusiv mentenanta, cerinte suplimentare de la client). Fiecare schimbare indeparteaza curba de 0 si genereaza un nou punct de "infant mortality". Prin urmare, curba defectelor nu este influentata de faptul daca cerintele sunt bine intelese sau de aplicarea sistematica a metodelor de testare eficiente`
   b. Utilizarea eficienta a instrumentelor CASE dedicate programarii poate reduce semnificativ costurile totale ale proiectului avand in vedere ca intr-un proiect software o parte semnificativa a costurilor sunt legate de activitatea de codare
   `FALS. In realitate, codarea nu este cea mai costisitoare parte dintr-un sistem SW. Partea de codare este doar 1/6 conform regulilor lui Brooks. Intr-adevar, instrumentele case pot reduce din costuri, dar nu intr-un mod semnificativ. Cea mai costisitoare parte o reprezinta testarea sistemului care valoreaza cam 1/2`
2. a. In procesul de dezvoltare "Waterfall" se face doar un singur tip de activitate la un moment dat, spre deosebire de cele iterative unde intr-o iteratie trebuie efectuate toate tipurile de activitati
   `ADEVARAT. Waterfall e un proces rigid, se stie de dinainte ce se va intampla la fiecare activitate si nu sunt permise schimbari pana nu se termina toate activitatile. In plus, totul se desfasoara secvential, fara sa ma pot intoarce inapoi, spre deosebire de cele iterative, unde sistemul creste cu fiecare iteratie si cerintele se pot schimba si reprioritiza in functie de dorintele clientului.`
   b. In procesul "Ratonal Unified Process", desi este permisa efectuarea mai multor tipuri de activitati in interiorul unei faze, se recomanda ca numarul de activitati desfasurate in paralel sa fie limitat.
   `FALS. RUP nu ne limiteaza cate activitati facem in paralel ci, din contra, incurajeaza paralelizarea activitatilor. RUP este un model hybrid, un proces iterativ si incremental cu 4 faze in care putem avea mai multe activitati in paralel, in functie de secventa noastra de milestone-uri.`
3. a. Use Case-urile de tip "Fish Level" se folosesc atunci cand vrem sa detaliem fiecare actiune principala din cadrul unui use-case de tip "Sea Level"
   `ADEVARAT. Un use-case de tip "Fish Level" este folosit pentru detalierea`

`actiunilor comune, individuale, iar cele "Sea Level" sunt folosite pentru interactiunile majore a actorilor cu sistemul. Din acest punct de vedere, putem folosi use-case de tip "Fish Level" pentru a detalia principalele actiuni din "Sea Level"`

b. Stim ca am descoperit toate Use-Case-urile "Sea Level", fiecare dintre acestea trebuind sa fie conectate cu cel putin un actor

`ADEVARAT. Un Use-Case "Sea Level" reperezinta o interactiune majora intre un actor si sistem. Prin urmare, actorul initiaza un comportament al sistemului si genereaza un use-case de acest tip

4. a. In tehnica CRC clasele sunt identificate pornind de la substantivele din descrierea use-case-urilor, iar responsabilitatile se identifica dintre verbele folosite in cadrul descrierii scenariilor

   `ADEVARAT. La baza tehnicii CRC stau USE-CASE-urile din care pot fi identificate clasele (pornind de la substantivele din descrierea use-case-urilor) si responsabilitatile/ operatiile/ metodele claselor (pornind de la verbele din descrierea use-case-urilor, deoarece acestea descriu operatiuni). EXTRA : Scopul intr-un proces de modelare CRC al claselor este sa ajungem in punctu in care putem sa "recream" intreaga descriere de use-case utilizand responsabilitatile si colaboratorii notati pe cardurile CRC`

   b. Relatia de compozitie este un caz special de agregare(relatie parte-intreg) in care indicam ca distrugerea clasei "intreg" atrage dupa sine si distrugerea obiectelor "parte" continute de clasa "intreg"

   `FALS. Intr-adevar, relatie de compozitie este un caz special de agregare(o forma mai stricta a relatiei de agregare), in care componentele nu pot exista fara obiectul de care depind. (De ex nu poti avea un buton de la un aparat, fara aparat). Distrugerea clasei "intreg" nu poate duce la distrugerea obiectelor componente in sine, ci doar distrugerea obiectului "intreg" poate duce la distrugerea obiectelor "parte".`

5. a. Intre specificatorul de acces "protected" si criteriul de modularitate al Protectiei exista o legatura stransa

   `FALS. Sunt doua lucruri diferite, specificatorul de acces "protected" permite ca atributele si metodele pe care este aplicat sa fie vizibile/accesibile doar din clasa respectiva sau din clasele care o mostenesc, pe cand criteriul de modularitate al Protectiei se refera, mai degraba, la faptul ca daca un modul este afectat, atunci eroarea nu se va propaga in multe module. Totusi, specificatorul de acces "protected" poate facilita implementarea criteriului, impreuna cu "private" si "package".`

   b. In stilul arhitectural "Repository" diferitele componente care proceseaza date sunt total independente unele de altele, cu exceptia faptului ca folosesc acelasi model de date

   `ADEVARAT. Stilul arhitectural "Repository" consta in faptul ca mai multe creiere interactioneaza cu datele din repo (deci folosesc acelasi model de date). Ele interactioneaza independent unii de ceilalti, deoarece producatorii si consumatorii de date sunt independenti (nu trebuie sa isi`

`faca griji de responsabilitati auxiliare, cum ar fi backup-ul si securitatea).`

6. a. Principalul motiv pentru care testarea de integrare de tip "Big Bang" nu este recomandata este acela ca spre deosebire de alte tehnici de testare de integrare, aceasta identifica mai putine defecte
   `FALS. Testarea de integrare de tip "Big Bang" presupune scrierea in intregime a codului, asamblarea tuturor componentelor sistemului si abia apoi se face testarea intregului sistem. Aceasta abordare nu e deloc recomandata, pentru ca izolarea erorilor este complicata, pot aparea erori care nu par a fi initial corelate si e dificil sa le rezolvi pentru ca te pierzi prin ele. Problema nu este ca identifica mai putine defecte, ci ca odata ce corectezi o eroare descoperita, apar altele.`
   b. Testarea ciclurilor pentru o functie cu doua cicluri imbricate(nested) implica scrierea a doua cazuri de test: unul care sa parcurga instructiunile din ciclul interior si un al doilea care sa testeze instructiunile din ciclul exterior
   `FALS. Testarea functiilor cu cicluri imbricate presupune parcurgerea acestora din interior spre exterior: Incepem cu ciclul interior si setam parametrii la minim din cel exterior si aplicam apoi cele mai simple reguli de testare pe ciclul interior, pastrand valorile minime la cel exterior. (skip la intregul loop; o singura trecere prin loop; doua treceri prin loop; N si (N - 1) treceri prin loop. Fixam mai apoi ciclul interior la o valoare tipica si ma concentrez asupra celui exterior aplicand aceleasi reguli de testare.`

Varianta 2 examen

1. a. Legile evolutiei software-ului enuntate de catre Lehmann ne spun ca nu exista nicio modalitate de a incetini declinul calitatii software-ului
   `FALS. Intr-adevar, declinul calitatii unui sistem are loc in mod continuu, datorita cerintelor clientilor, care se modifica in mod constant si a schimbarii de pe piata. Un factor decisiv este, insa, mentenanta riguroasa si adaptarea sistemului in conformitate cu aceste schimbari cotidiene, modalitate prin care se poate incetini declinul`
   b. Printr-o proiectare riguroasa, implementarea software-ului poate fi transformata intr-o activitate aproape perfect partitionata
   `FALS. O proiectare riguroasa ar facilita partitionarea task-urilor necesare implementarii software-ului, dar nu poate garanta perfectiunea. Limita este data de numarul de subtask-uri independente al proiectului, pentru ca intr-un proiect este nevoie de comunicare si relatii complexe.`
2. a. Procesul de dezvoltare Extreme Programming spune ca majoritatea efortului trebuie investit in programare si mai putin in alte activitati conexe, cum ar fi captarea cerintelor sau testare, intrucat, pana la urma, produsul software propriu-zis este creat prin programare.

`FALS. Extreme Programming este cea mai buna metoda Agile cu o abordare dusa la extrem a metodei iterative. O versiune noua apare in fiecare zi/noapte, iar testarea are loc la fiecare build. Cel mai mult efort este investit, prin urmare, in testare(1/2 conforma legii lui Brooks), deoarece testarile au loc f des pentru a putea da accept la un build. In plus, intr-un proiect Agile se pune accent si pe o organizare eficienta a muncii, prin sprint-uri.`

b.Desi in procesele de dezvoltare iterative si incrementale trebuie uneori schimbate sau chiar rescrise complet fragmente semnificative de cod ce au fost scrise in iteratiile anterioare, acestea nu reprezinta o pierdere, intrucat schimbarile sunt inevitabile.

`ADEVARAT. Schimbarile sunt inevitabile(cerintele se schimba, procesele de dezvoltare iterativa se bazeaza pe flexibilitate), vor fi momente in care fragmente semnificative de cod scrise anterior vor trebui rescrise/schimbate, insa aceasta nu este o pierdere de timp, ci mai mult o perfectionare a codului deja existent(de ex daca este nevoie sa facem modificari la motorul unei masini, poate parea o pierdere de timp, insa este, in definitiv, un castig, deoarece obtin un produs mai buna). In plus, este mult mai bine sa rescrii codul, decat sa peticesti/corectezi un cod scris prost`

3. a. Tehnica de analiza a cerintelor folosind Use-Case-uri este specifica procesului de dezvoltare "Waterfall" pentru ca aici cerintele trebuie analizate riguros la inceputul proiectului

   `FALS. Tehnica de analiza a cerintelor folosind Use-Case-uri nu este specifica doar procesului de dezvoltare "Waterfall". Un Use-Case este un set de scenarii care descriu cerintele clientului, nefiind specific unui singur proces de dezvoltare. Intr-adevar, in Waterfall si in orice proiect cerintele trebuie analizate riguros la inceput, cu atat mai mult daca procesul nu este favorabil schimbarilor in timpul iteratiilor(Waterfall e un proces riguros).`

   b. Doi sau mai multi actori nu pot interactiona/nu pot fi asociati cu acelasi use case, pentru ca ar insemna ca sunt redundanti.

   `FALS. Doi sau mai multi actori pot interactiona/pot fi asociati cu acelasi use case, atata timp cat au si alte functionalitati decat cele comune. De ex, daca luam in considerare un sistem cu Campus-ul virtual, atat studentul cat si profesorul pot accesa use case-ul de LOGIN, insa doar profesorul poate accesa use-case-ul de ATRIBUIE_NOTE.`

4. a. Intr-o diagrama de secventa UML, toate obiectele care sunt instante ale aceleiasi clase trebuiesc "comasate" intr-un singur dreptunghi, adica nu este admis sau recomandat sa reprezentam fiecare obiect ca un dreptunghi separat, pentru ca s-ar incarca prea mult diagrama.

   `FALS. Diagrama de secventa ne arata viata si interactiunile dintre obiecte, nu are niciun sens sa comasam intr-un singur element doua obiecte diferite. Fiecare obiect poate avea un comportament diferit, chiar daca 2 sau mai`

`multe pot fi instante ale aceleasi clase.`

b. Pentru a evita problema proliferarii claselor prin modelarea ca si clase a entitatilor externe sistemului, trebuie sa tinem cont ca sunt clase doar acele entitati care apeleaza alte entitati(clase) din sistem.

`FALS. Intr-adevar, problema proliferarii claselor presupune evitarea modelarii entitatilor externe(sistemului) ca si clase, adica este important sa nu transformam inutil operatiile in clase. Insa, o clasa este cea care este apelata, nu cea care apeleaza(transmite mesaj).`

5. a. Un corp de mobila modular care vine impachetat pe bucati si care trebuie asamblat(gen IKEA) a fost proiectat urmarit in special criteriul de modularitate al compozabilitatii.

   `FALS. Un corp de mobila modular este proiectat in asa fel incat, asambland piesele/diferitele componente dupa un anumit plan/ iinstructiuni sa putem obtine acelasi produs final, lucru ce este urmarit de criteriul decompozabilitatii. De altfel, criteriul compozabilitatii se refera, mai degraba, la posibilitatea de a ajunge la output-uri diferite, pornind de la aceleasi piese(De ex, galetile de LEGO vechi, care nu veneau cu instructiuni si puteai sa construiesti orice voiai).`

   b .Importanta criteriilor si regulilor de modularitate este cu atat mai mare cu cat anvergura sistemului software proiectat este mai mare, cu alte cuvinte: importanta modularizarii este proportionala cu dimensiunea sistemului software.

   `ADEVARAT. Intr-adevar, criteriile de modularitate ajuta mult in gestionarea complexitatii unui proiect software, care inherent este mai mare in cazul unui sistem de anvergura mai mare. Respectand criteriile, schimbarile pot fi efectuate mai eficient, nefiind necesara modificari majore in intreg sistemul pentru o singura modificare.` Pe de alta parte, poate fi adus si argumentul ca este importanta respectarea criteriilor de modularitate chiar si in cazul unui sistem software de anvergura mai mica, pentru a asigura scalabilitatea pe viitor. Implicit, proiectele software sunt supuse multor schimbari prin definitie si trebuie sa fie proiectate astfel incat sa suporte eficient scalarea.`

6. a. Daca dorim sa verificam cat mai timpuriu principalele puncte de control si decizie din sistem, vom alege testarea Bottom-Up.

   `FALS. Testarea de tip Bottom-Up incepe cu tesatea modulelor low-level/frunza (se utilizeaza cluster-e in acest tip de testare). Modulele mai inalte/neimplementate sunt inlocuite de drivere. Pentru a testa timpuriu principalele puncte de control si decizie vom alege, in schimb, testarea Top-Down, care inlocuieste modulele low-level cu stub-uri, dar testeaza modulele principale de cele mai multe ori(prin urmare si principalele puncte de control si decizie).`

   b. Valoarea complexitatii ciclomatice a unei functii nu este influentata de numarul de instructiuni de ciclare(for, while), deoarece singurele instructiuni care incrementeaza valoarea complexitatii ciclomatice sunt cele de decizie de tip if..else.

   `FALS. Si instructiunile de ciclare au de evaluat o conditie de oprire,`

`altfel s-ar crea un loop infinit. Prin urmare, fiecare instructiune de ciclare va contribui la complexitatea ciclomatica, alaturi de cele de decizie if-else(de ex avem un for(i = 0; i < 10; i++), se va verifica i < 10 la fiecare bucla, DAR complexitatea ciclomatica va creste doar cu 1).`

Curs 1

1. Forma reala a curbei ratei defectarii software-ului este datorata faptului ca cerintele nu sunt intelese clar de la bun inceput, ceea ce duce la necesitatea de schimbari continue asupra sistemului.
   `FALS. Forma reala a curbei ratei defectarii software-ului este datorata schimbarilor constante prin care trece un sistem pentru a se adapta cerintelor in continua schimbare. Schimbarile apar chiar daca cerintele au fost sau nu intelese clar de la bun inceput, deoarece fiecare sistem trebuie sa fie supus schimbarilor pentru a ramane relevant.`
2. 50% din timp si costuri ar trebui alocate testarii, dar in multe cazuri procentajul alocat testarii este mai mic.
   `ADEVARAT. Testarea reprezinta cea mai costisitoare activitate, 1/2 din tot sistemul. Intr-adevar, in multe cazuri, nu se aloca acest procentaj testarii, insa intr-un final tot se ajunge la acest procent, deoarece cresterea in complexitate a sistemului presupune o nevoie mai mare de testare.`
3. Curba reala a evolutiei in timp a ratei de defecte (Failure Rate) diferta fata de cea ideala in principal din cauza lipsei de experienta a programatorilor.
   `FALS. Forma curbei reale nu este influentata de lipsa experientei programatorilor, ci de schimbarile care se impun odata cu aparitia cerintelor noi. Fiecare schimbare implementata pentru lucrari de mentenanta, cerinte, modificari de soft duce la aparitia unui spike (infant mortality) si departarea de axa Ox a curbei. Lipsa de experienta a programatorilor poate influenta alte aspecte din producerea unui sistem.`
4. Legile lui Lehman ale evolutiei software-ului nu se aplica la sisteme ale caror cerinte sunt intelese perfect de la bun inceput, pentru ca astfel de sisteme nu au niciun motiv sa evolueze.
   `FALS. Legile lui Lehman se aplica tuturor sistemelor software, indiferent de modul cum au fost intelese cerintele la inceput. Sistemele software sunt obligate sa evolueze pentru a ramane relevante(in viata), deoarece ele devin obsolete daca nu se adapteaza schimbarilor cerute, daca nu sunt intretinute.`
5. Legile evolutiei software-ului enuntate de catre Lehmann ne spun ca nu exista nicio modalitate de a incetini declinul calitatii software-ului
   `FALS. Intr-adevar, declinul calitatii unui sistem are loc in mod continuu, datorita cerintelor clientilor, care se modifica in mod constant si a schimbarii de pe piata. Un

factor decisiv este, insa, mentenanta riguroasa si adaptarea sistemului in conformitate cu aceste schimbari cotidiene, modalitate prin care se poate incetini declinul

6. Printr-o proiectare riguroasa, implementarea software-ului poate fi transformata intr-o activitate aproape perfect partitionata
   `FALS. O proiectare riguroasa ar facilita partitionarea task-urilor necesare implementarii software-ului, dar nu poate garanta perfectiunea. Limita este data de numarul de subtask-uri independente al proiectului, pentru ca intr-un proiect este nevoie de comunicare si relatii complexe.`
7. Curba defectelor poate fi facuta sa tinda in timp spre zero daca cerintele sunt bine intelese de la inceput si daca se aplica sistematic metode de testare eficiente
   `FALS. Curba defectelor poate fi facuta sa tinda spre 0 doar intr-un caz ideal, lucru inexistent de altfel. In realitate, curba defectelor se indeparteaza de 0 pe masura ce se aplica schimbari sistemului SW (inclusiv mentenanta, cerinte suplimentare de la client). Fiecare schimbare indeparteaza curba de 0 si genereaza un nou punct de "infant mortality". Prin urmare, curba defectelor nu este influentata de faptul daca cerintele sunt bine intelese sau de aplicarea sistematica a metodelor de testare eficiente.`
8. Utilizarea eficienta a instrumentelor CASE dedicate programarii poate reduce semnificativ costurile totale ale proiectului avand in vedere ca intr-un proiect software o parte semnificativa a costurilor sunt legate de activitatea de codare
   `FALS. In realitate, codarea nu este cea mai costisitoare parte dintr-un sistem SW. Partea de codare este doar 1/6 conform regulilor lui Brooks. Intr-adevar, instrumentele case pot reduce din costuri, dar nu intr-un mod semnificativ. Cea mai costisitoare parte o reprezinta testarea sistemului care valoreaza cam 1/2`
9. Criza software din anii '60 a fost cauzata de hardware-ul insuficient de puternic fata de complexitatea excesiva a software-ului din acel moment.
   `FALS. Criza software din anii '60 a fost cauzata de cresterea mult prea rapida a hardware-ului(computere puternice), iar programatorii nu erau obisnuiti cu un asemenea salt in complexitate(Nu erau pregatiti sa scrie programe complexe care fie corecte, usor de inteles si usor de verificat/testat). Din aceste motive a fost nevoie sa se dezvolte procese pentru a gestiona software-ul.`
10. Cunoasterea perfecta a limbajului de programare in care se va implementa un proiect de mari dimensiuni este o conditie necesara si suficienta pentru terminarea cu succes a proiectului.
    `FALS. Partea de codare reprezinta, conform legilor lui Brooks, doar 1/6 din intreg procesul de dezvoltare al unui soft si, desi este important si constituie un avantaj sa cunosti bine limbajul de programare utilizat in proiect, nu este o conditie suficienta pentru terminarea cu succes a proiectului.Exista si alte parti care sunt necesare, cea mai importanta fiind testarea software-ului(1/2), dar si planificarea(1/3).`

11. Intr-un sistem cu o modularitate foarte buna, legea lui Brooks nu se aplica, deoarece construirea sistemului e o activitate perfect partitionata.
`FALS. Construirea unui sistem nu este o activitate perfect partitionata, pentru ca exista diferite tipuri de task-uri: cu interdependente sau nu. Nu este suficient sa adaugam mai multi oameni intr-un proiect aflat in intarziere, deoarece acesti noi oameni vor trebui sa inteleaga procesul si abia apoi sa lucreze, pe langa faptul ca limita de oameni dintr-un proiect este data de nr de task-uri independente care exista, altfel ar astepta unii dupa altii.Acest aspect este atins si in Mythical Man-Month, care ne demonstreaza ca nu putem cuantiza liniar munca intr-un proiect software.`
12. Testarea ocupa 25% din costul total al dezvoltarii unui sistem software.
`FALS. Testarea ocupa 50% din costul total al dezvoltarii unui sistem software conform Legilor lui Brooks, deoarece 1/4 merge spre testarea componentelor si 1/4 merge pe testarea sistemului. Chiar daca nu este alocat initial 50% pentru testare, tot se va ajunge la acest procent sau chiar mai mult, pe masura ce proiectul creste in complexitate d.p. cu nr de teste necesare`

Curs 2

1. Conceptul de time boxing spune ca sarcinile alocate intr-o anumita iteratie trebuiesc terminate in cadrul iteratiei in cauza.
`FALS. Conceptul de time boxing presupune alocarea unui interval de timp fix pentru fiecare iteratie, pe care nu ai voie sa-l modifici, indiferent de ce s-ar intampla. De ex, daca nu ai reusit sa implementezi toate feature-urile stabilite pentru o iteratie, scoti din ele, dar nu modifici deadline-ul. Task-urile nefinalizate pot fi mutate pentru urmatoarele iteratii, fiind un exercitiu bun pentru prioritizarea cerintelor`
2. Daca lucram cu procese de dezvoltare agile, asta inseamna ca nu trebuie sa intelegem clar cerintele sistemului.
`FALS. Indiferent de procesul de dezvoltare utilizat, cerintele sistemului trebuie intelese pentru a finaliza proiectul. In acelasi timp, intr-un proces de dezvoltare Agile, proces flexibil, cerintele sistemului se pot modifica de la o iteratie la alta, deci cerintele trebuie analizate si intelese la fiecare iteraatie, pentru a putea face modificarile necesare.`
3. In procesul de dezvoltare SCRUM, Burndown Chart-ul ne arata evolutia efortului de-a lungul intregii istorii a proiectului, mai exact ofera informatii despre Sprint-urile deja incheiate, precum si numarul de task-uri finalizate pana in prezent in intregul proiect.
`FALS. Burndown Chart-ul este un grafic care ne arata efortul si timpul ramas, pana la finalizarea sprint-ului curent. In niciun caz nu ne arata informatii despre istoria intregului proiect. De asemenea, ne arata si cate task-uri au mai ramas.`

4. O problema importanta a proceselor de dezvoltare iterative este aceea ca adesea trebuie modificat codul care a fost scris intr-o iteratie anterioara si uneori anumite portiuni de cod trebuie chiar sterse, ceea ce reprezinta o pierdere/risipa
`FALS. Intr-adevar se poate intampla ca adesea in procesele de dezvoltare iterative sa fie nevoie ca fragmente mari de cod sa fie rescrise/modificate, insa nu este un dezavanntaj caci este mai bine sa rescrii o bucata de cod de la 0 decat sa incerci sa o peticesti incat sa se conformeze la noile cerinte.`
5. Procesul de dezvoltare Extreme Programming spune ca majoritatea efortului trebuie investit in programare si mai putin in alte activitati conexe, cum ar fi captarea cerintelor sau testare, intrucat, pana la urma, produsul software propriu-zis este creat prin programare.
`FALS. Extreme Programming este cea mai buna metoda Agile cu o abordare dusa la extrem a metodei iterative. O versiune noua apare in fiecare zi/noapte, iar testarea are loc la fiecare build. Cel mai mult efort este investit, prin urmare, in testare(1/2 conforma legii lui Brooks), deoarece testarile au loc f des pentru a putea da accept la un build. In plus, intr-un proiect Agile se pune accent si pe o organizare eficienta a muncii, prin sprint-uri.`
6. .Desi in procesele de dezvoltare iterative si incrementale trebuie uneori schimbate sau chiar rescrise complet fragmente semnificative de cod ce au fost scrise in iteratiile anterioare, acestea nu reprezinta o pierdere, intrucat schimbarile sunt inevitabile.
`ADEVARAT. Schimbarile sunt inevitabile(cerintele se schimba, procesele de dezvoltare iterativa se bazeaza pe flexibilitate), vor fi momente in care fragmente semnificative de cod scrise anterior vor trebui rescrise/schimbate, insa aceasta nu este o pierdere de timp, ci mai mult o perfectionare a codului deja existent(de ex daca este nevoie sa facem modificari la motorul unei masini, poate parea o pierdere de timp, insa este, in definitiv, un castig, deoarece obtin un produs mai buna). In plus, este mult mai bine sa rescrii codul, decat sa peticesti/corectezi un cod scris prost`
7. In procesul de dezvoltare "Waterfall" se face doar un singur tip de activitate la un moment dat, spre deosebire de cele iterative unde intr-o iteratie trebuie efectuate toate tipurile de activitati
`ADEVARAT. Waterfall e un proces rigid, se stie de dinainte ce se va intampla la fiecare activitate si nu sunt permise schimbari pana nu se termina toate activitatile. In plus, totul se desfasoara secvential, fara sa ma pot intoarce inapoi, spre deosebire de cele iterative, unde sistemul creste cu fiecare iteratie si cerintele se pot schimba si reprioritiza in functie de dorintele clientului.`
8. In procesul "Ratonal Unified Process", desi este permisa efectuarea mai multor tipuri de activitati in interiorul unei faze, se recomanda ca numarul de activitati desfasurate in paralel sa fie limitat.

`FALS. RUP nu ne limiteaza cate activitati facem in paralel ci, din contra, incurajeaza paralelizarea activitatilor. RUP este un model hybrid, un proces iterativ si incremental cu 4 faze in care putem avea mai multe activitati in paralel, in functie de secventa noastra de milestone-uri.`

9. Procesele de dezvoltare Agile se numesc asa pentru ca atunci cand le folosim, dezvoltarea intregului sistem dureaza garantat mai putin decat daca folosim alte procese.
   `FALS. Desi intr-un proces de dezvoltare Agile, frecventa de livrare a unui working product catre client este mai mare, nu inseamna ca dezvoltarea intregului sistem va lua garantat mai putin. Principiile Agile nu au ca scop un proces de dezvoltare mai rapid, ci unul cu un raspuns mai bun(mai agil) la modificarile cerintelor si o comunicare mai buna cu clientii, constituind un proces flexibil.`
10. In procesele de dezvoltare iterative se stabileste pentru fiecare iteratie un set de functionalitati care trebuiesc adaugate, iar daca se constata ca timpul prevazut pentru iteratie nu este suficient pentru implementarea intregului set de functionalitati, se prelungeste corespunzator durata iteratiei si se creste preventiv si durata urmatoarei iteratii.
    `FALS. Intr-adevar, la inceputul fiecarei iteratii se stabileste un set de features care trebuie implementate, insa daca nu se reuseste implementarea tuturor timpul prevazut nu se prelungeste. Acesta este conceptul de TIME BOXING, care presupune stabilirea unui interval de timp fix pentru fiecare iteratie, care nu se modifica indiferent de ce se intampla. Feature-urile neimplementate se intorc in project backlog si pot fi reprioritizate sau nu si implementate/sau nu in iteratiile urmatoare.`
11. In procesul de dezvoltare SCRUM, daca functionalitatile prevazute pentru un Sprint nu pot fi terminate intr-un timp alocat, Sprint-ul poate fi prelungit, dar nu mai mult de cateva zile.
    `FALS. Scrum este un proces de dezvoltare Agile, in cadrul caruia iteratiile se numesc Sprint-uri si respecta principiul de TIME BOXING, adica pentru fiecare sprint este alocat un interval de timp fix, nemodificabil, indiferent de ce se intampla. Daca functionalitatile dintr-un sprint backlog nu apuca sa fie implementate, timpul alocat sprint ului nu va fi prelungit sub nicio forma si functionalitatile se vor intoarce in product backlog unde vor fi reprioritizate(daca este cazul) si reevaluate de catre echipa pentru a fi introduse in sprint-urile urmatoare.`
12. You have to develop a student management system for a university. The university is chaotic, with very radical, frequent and fast changes with respect to the student organisation policies. In this context, the best idea is to use a Waterfall development system to build the required system.
    `FALS. Waterfall e un proces de dezvoltare destul de rigid in fata schimbarilor. Prin urmare, acesta nu ar fi deloc potrivit pentru un asemenea`

`management system si ar trebui aleasa o modalitate iterativa de dezvoltare, care sa raspunda mai rapid la cerintele care se schimba des.`

13. Procesele de dezvoltare Agile sunt recomandate in cazul in care proiectele la care lucreaza echipele de programatori sunt distribuite geografic.
    `FALS. Fiind niste procese iterative, procesele de dezvoltare Agile nu sunt potrivite in cazul in care proiectele la care lucreaza echipele de programatori sunt distribuite geografic, deoarece necesita comunicare, interactiune intre membrii echipei pentru a monotoriza progresul intr-o iteratie si a isi efectua meeting-urile.Intr-un proces Agila bazat pe Extreme Programming, eficienta poate scadea chiar si daca echipa de programatori este impartita pe mai multe etaje din aceeasi cladire.`
14. Procesul de dezvoltare Waterfall este recomandat in majoritatea proiectelor, intrucat, datorita structurii sale rigide, poate constrange clientul sa descopere, inca de la inceput, toate posibilele cauze de schimbare din sistem.
    `FALS. Tocmai din cauza faptului ca este un proces de dezvoltare rigid, Waterfall este recomandat in putine cazuri, deoarece nu este usor adaptabil la cerinte noi, care pot aparea oricand in dezvoltarea unui sistem/proiect. Aceasta constrangere nu poate sa il determine pe client sa descopere toate posibilele cauze de schimbare de sistem, deoarece multe din acestea nu pot fi predictibile, asa ca este necesar sa folosim un proces de dezvoltare iterativ de multe ori. Software ul e controlabil, dar nu predictibil.`
15. Ce ati alege dintre Waterfall si Extreme Programming/Agile?
    `- CAZ 1:` - Trebuie sa implementam de la 0 un sistem complex(cca. 1 milion de linii de cod) pentru gestionarea personalului si a salariilor, care sa inlocuiasca sistemul existent in prezent, fara nicio modificare de cerinte.
    - **WATERFALL**
    `` Cerintele sunt bine cunoscute de la inceput(fiind vorba despre reimplementare) si bine intelese, si nu exista riscul de a aparea schimbari in aceste cerinte. Astfel, procesul Waterfall se poate derula fara probleme, nefiind nevoie de capacitatea de a raspunde rapid la schimbari.
    - `CAZ 2: -` Aveti de construit un sistem pentru plata taxelor si impozitelor pentru tara imaginara Ainamor cu o legislatie haotica si in permanenta modificare.
      - **AGILE/Extreme Programming**
      `Este vorba de implementarea unui proiect cu cerinte care sunt in continua schimbare, asa ca trebuie sa folosim un proces de dezvoltare capabil sa raspunda rapid la schimbari dese.` Agile, fiind un proces iterativ, poate sa raspunda rapid la schimbari, iar faptul ca este incremental inseamna ca aceste raspunsuri ajung la client in timp scurt.
      `

Curs 3

1. Un sistem nu poate fi actor pentru un alt sistem, pentru ca acest lucru ar implica accesul la informatiile sistemului mare, ceva ce implica incalcarea incapsularii.
   `FALS. Un sistem poate fi actor pentru un alt sistem, atata timp cat este extern sistemului, adica nu are acces la codul sistemului neactor(deoarece interactioneaza cu interfata). Prin urmare, nu se incalca principiul incapsularii`
2. In diagramele UML de use case-uri, relatiile `<<extends>>` si `<<include>>` sunt foarte asemanatoare si pot fi folosite interschimbabil, intrucat ambele sunt folosite pentrui a da "factor comun" in cadrul unei anumite functionalitati.
   `FALS. Cele doua relatii nu sunt asemanatoare si nu pot fi folosite interschimbabil. Relatia de <<include>> este cea care este folosita pentru a da factor comun unei functionalitati intre mai multe useCase-uri(cumperiUnTicket, cumperiMultiCard -> ambele implica CollectMoney). Pe de alta parte, <<extends>> este folosita pentru a trata un caz exceptional care poate aparea in cazul unui use case(dar nu este obligatoriu sa se intample; ex: platesti -> outOfMoney)`
3. Doi sau mai multi actori nu pot interactiona/nu pot fi asociati cu acelasi use case, pentru ca ar insemna ca sunt redundanti.
   `FALS. Doi sau mai multi actori pot interactiona/pot fi asociati cu acelasi use case, atata timp cat au si alte functionalitati decat cele comune. De ex, daca luam in considerare un sistem cu Campus-ul virtual, atat studentul cat si profesorul pot accesa use case-ul de LOGIN, insa doar profesorul poate accesa use-case-ul de ATRIBUIE_NOTE.`
4. Use case-urile de tip Fish Level se folosesc atunci cand vrem sa detaliem fiecare actiune principala din cadrul unui use case de tip Sea Level.
   `ADEVARAT. Un use-case de tip "Fish Level" este folosit pentru detalierea actiunilor comune, individuale, iar cele "Sea Level" sunt folosite pentru interactiunile majore a actorilor cu sistemul. Din acest punct de vedere, putem folosi use-case de tip "Fish Level" pentru a detalia principalele actiuni din "Sea Level"`
5. Stim ca am descoperit toate use case-urile din sistem atunci cand toate cerintele functionale sunt acoperite de use case-uri Sea Level, fiecare dintre acestea trebuind sa fie conectate cu cel putin 1 actor.
   `ADEVARAT. Un Use-Case "Sea Level" reperezinta o interactiune majora intre un actor si sistem. Prin urmare, actorul initiaza un comportament al sistemului si genereaza un use-case de acest tip`
6. Use case-ul "Retrage bani de la ATM" este intr-o relatie de tip `<<include>>` cu use case-ul "Retrage bani de la ATM ramas fara cash" pentru ca primul e mai cuprinzator.
   `FALS. Primul use case este un caz de baza, general, iar al doilea pare a fi un caz exceptional, care nu se intampla de fiecare data deci nu poate fi intr-o relatie de include cu primul ci, mai degraba, intr-o relatie de extends.`

7. In tehnica FAST de analiza a cerintelor, fiecare participant trebuie sa construiasca cate 4 liste, independent de ceilalti si fara presiunea ca listele sa fie complete.
   `ADEVARAT. In tehnica FAST, fiecare participant la intalnire vine cu cate 4 liste despre : obiecte, servicii, constrangeri si criterii de performanta. Aceste liste sunt sumare si necesare pentru a alcatui o lista cu mai multe puncte de vedere, fara a elimina nicio informatie care nu este duplicat.`
8. It is possible for a person(from the real world) to have the role of more than one actor of the system.
   `ADEVARAT. O persoana poate sa indeplineasca mai multe roluri. Sa zicem ca are o functie care indeplineste mai multe responsabilitati, iar aceste responsabilitati vor fi, de fapt, actorii. Invers, nu este indicat sa ne alegem actorii dupa functia unei persoane. De exemplu, o persoana poate fi atat administratorul unui magazin, cat si client`
9. An `<<extends>>` relation like in the picture means that B use case inherits all the interactions from the B use case and, additionally, B adds some supplementary interaction steps to the inherited one.
   `N-avem poza`
10. Tehnica de analiza a cerintelor folosind Use-Case-uri este specifica procesului de dezvoltare "Waterfall" pentru ca aici cerintele trebuie analizate riguros la inceputul proiectului
    `FALS. Tehnica de analiza a cerintelor folosind Use-Case-uri nu este specifica doar procesului de dezvoltare "Waterfall". Un Use-Case este un set de scenarii care descriu cerintele clientului, nefiind specific unui singur proces de dezvoltare. Intr-adevar, in Waterfall si in orice proiect cerintele trebuie analizate riguros la inceput, cu atat mai mult daca procesul nu este favorabil schimbarilor in timpul iteratiilor(Waterfall e un proces riguros).`

Curs 4

1. Pentru a evita problema proliferarii datelor, ar trebui sa implementam clasa Family ca mai jos.

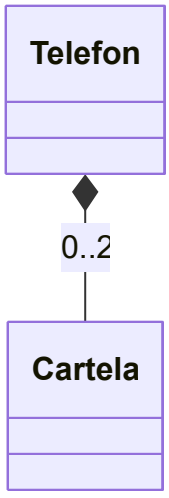
   ```
   ![[Pasted image 20250620003146.png]]
   ```

   - **ADEVARAT.**
     `La modul cel mai abstract, o familie este formata din doua persoane ca si parinti si o lista de alte persoane ca si copii.` In cazul de fata, daca difera functionalitatea parintilor versus copii, prin polimorfism s-ar putea ca parintii sa fie clase separate, mostenitoare ale `Person`, analog si pentru copii.
     `` Implementarea aceasta mai generala permite si un arbore genealogic mai complex, avand in vedere ca unii copii la randul lor pot fi parinti.
     !!! De avut grija la examen ce poza da

2. Daca avem diagrama de clasa UML explicita pentru un sistem, putem deduce diagramele de secventa pentru acel sistem, adica toate interactiunile posibile dintre obiecte.
   `FALS. Diagrama de clasa UML reprezinta structura sistemului, adica ne arata ce atribute si metode contine clasa respectiva. Diagrama de secventa nu poate fi dedusa doar din aceste informatii, deoarece ea se refera la comportamentul instantelor de clase dintr-un sistem(obiectele). Acestea sunt construite pe baza codului sursa, reprezentand functionarea acestuia in sine raportata la timp.`

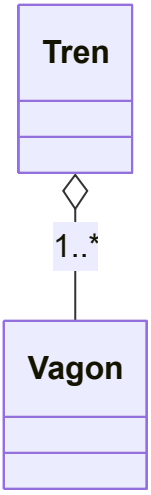
!! Cele de secventa sunt costisitoare, dar merita investit in ele

3. Diagramele UML de secventa sunt folosite ca punct de plecare pentru sesiunile de CRC Cards, in urma carora se construiesc diagramele de clasa.
   `FALS. Sesiunile de CRC Cards sunt use case driven si nu folosesc diagramele uml de secventa ca punct de plecare. Modelarea CRC e o tehnica folosita pentru a identifica clasele, responsabilitatile si colaboratorii unei clase.`
4. Intr-o diagrama de secventa UML, toate obiectele care sunt instante ale aceleiasi clase trebuiesc "comasate" intr-un singur dreptunghi, adica nu este admis sau recomandat sa reprezentam fiecare obiect ca un dreptunghi separat, pentru ca s-ar incarca prea mult diagrama.
   `FALS. Diagrama de secventa ne arata viata si interactiunile dintre obiecte, nu are niciun sens sa comasam intr-un singur element doua obiecte diferite. Fiecare obiect poate avea un comportament diferit, chiar daca 2 sau mai multe pot fi instante ale aceleasi clase.`
5. Relatia de mai jos modeleaza cazul unui telefon dual-sim cu cartele care pot fi oricand utilizate in alt telefon.



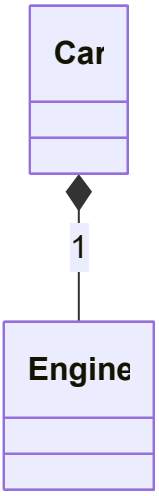
   `FALS. Avem o relatie de compozitie, lucru care sugereaza ca nu poate exista o cartela in afara obiectului de tip telefon dual sim, adica la un obiect de tip telefon normal.`

6. Pentru a evita problema proliferarii claselor prin modelarea ca si clase a entitatilor externe sistemului, trebuie sa tinem cont ca sunt clase doar acele entitati care apeleaza alte entitati(clase) din sistem.
   `FALS. Intr-adevar, problema proliferarii claselor presupune evitarea modelarii entitatilor externe(sistemului) ca si clase, adica este important sa nu transformam inutil operatiile in clase. Insa, o clasa este cea care este apelata, nu cea care apeleaza(transmite mesaj)`
7. In tehnica CRC, clasele sunt identificate pornind de la substantivele din descrierea use case-urilor, iar responsabilitatea se identifica prin verbele folosite in cadrul descrierii scenariilor.
   `ADEVARAT. La baza tehnicii CRC stau USE-CASE-urile din care pot fi identificate clasele (pornind de la substantivele din descrierea use-case-urilor) si responsabilitatile/ operatiile/ metodele claselor (pornind de la verbele din descrierea use-case-urilor, deoarece acestea descriu operatiuni). EXTRA : Scopul intr-un proces de modelare CRC al claselor este sa ajungem in punctu in care putem sa "recream" intreaga descriere de use-case utilizand responsabilitatile si colaboratorii notati pe cardurile CRC`
8. Relatia de compozitie este un caz special de agregare(relatie parte-intreg) in care indicam ca distrugerea clasei "intreg" atrage dupa sine si distrugerea obiectelor "parte" continute de clasa "intreg"
   `FALS. Intr-adevar, relatie de compozitie este un caz special de agregare(o forma mai stricta a relatiei de agregare), in care componentele nu pot exista fara obiectul de care depind. (De ex nu poti avea un buton de la un aparat, fara aparat). Distrugerea clasei "intreg" nu poate duce la distrugerea obiectelor componente in sine, ci doar distrugerea obiectului "intreg" poate duce la distrugerea obiectelor "parte".`
9. Se da urmatoarea diagrama de clasa. Explicati:



   `Explicam diagrama ca nuj ce cere`
   `Avem o relatie de agregare adica, trenul agrega mai multe vagoane(1, 2, etc oricate)-> HAS A / is part of. Existenta obiectelor de tip vagon nu depinde de existenta obiectului de tip tren, adica vagoanele pot sa existe independent si se pot afla in relatie cu un alt obiect tren.`

10. Ar trebui modelate in sistem doar clasele care transmit mesaje, nu si cele care primesc mesaje.
    `FALS Clasele nu pot transmite mesajele, pe clase doar se apeleaza metode(primesc mesaje), conform rezolvarii Problemei Proliferarii Claselor.`
11. Relatia se interpreteaza astfel: clasa Car contine clasa Engine, mai precis cand se distruge clasa Car, se distruge si clasa Engine.



    `FALS. In imaginea de mai sus, avem o relatie de compozitie intre motor si masina, semnificata printr-un diamant plin. Aceasta este o relatie de tip "PART-OF", conform careia obiectele parte pot fi folosite de un singur obiect intreg, iar distrugerea obiectelor parte are loc doar odata cu distrugerea obiectelor intreg. Afirmatia este falsa, deoarece nu se distruge clasa in sine, ci obiectul, instanta a clasei. => daca se distruge un obiect de tip car se distruge si obiectul de tip engine care se afla in relatie de compozitie cu acesta.`
12. Scenariile aferente use case-urilor si diagramelor de secventa UML sunt doua modalitati alternative de modelare a interactiunii intre clase. Cu alte cuvinte, daca dorim sa observam cum colaboreaza clasele, putem folosi fie use case-uri, fie diagrame de secventa.
    `FALS. Scenariile aferente use-case-urilor surprind interactiunile dintre actori si siste, insa nu se poate observa pe baza lor comportamentul claselor, asafar nu este modalitate de modelare a interactiunii intre clase. De pe baza use-case-urilor putem apela la CRC card modelling-> care ar fi ok. De altfel, diagramele de secventa UML descriu obiectele si interactiunile dintre ele, cum se apeleaza metodele pe ele.`
13. Un actor nu poate introduce informatii in sistemul modelat, ci poate doar extrage.
    `FALS. Actorul reprezinta rolul jucat de un user in cadrul interactiunii cu sistemul. Un actor poate ori sa introduca informatii, ori sa extraga, ori ambele. Un exemplu este adaugarea unei note intr-o aplicatie de tip Campus Virtual.`
14. In modelarea cerintelor se poate aplica urmatoarea regula: aproape orice substantiv este un actor si aproape orice verb este un use case.
    `FALS. Orice verb poate reprezenta o responsabilitate/ o operatie, dar nu un`

`use case. UseCase-urile sunt un set de scenarii care prezinta interactiunea dintre actor si sistem si includ verbe si substantive. Actorul reprezinta un rol jucat de un user in cadrul interactiunii cu sistemul, dar nu orice substantiv poate fi un actor. Un substantiv poate fi o clasa, un atribut, etc.`

15. The CRC Cards model of a system helps us to start building class diagrams and sequence diagrams for that system.
`ADEVARAT. Utilizand modelul CRC, obtinem clasele, responsabilitatile acestora, si colaborarile acestora.Din aceste informatii, putem incepe sa construim diagramele de secventa si de clasa. Folosind card-urile claselor si colaboratorii acestora putem determina clasele si relatiile dintre acestea, iar folosind si responsabilitatile, putem determina campurile si operatiile/metodele din clase. Avand aceste informatii, putem incepe sa construim atat diagrama de clasa UML, cat si diagrama de secventa UML.`

Curs 5

1. Stilul arhitectural Pipes&Filters este avantajos din punct de vedere al timpului.
`FALS. Acest stil este format din filtre, care sunt independente unul fata de celalalt, fiecare avand stare separata. Abstractizand, tot ce face un filtru este sa transforme datele de intrare in date de iesire, dupa un anumit criteriu. Acest lucru face mentenanta si schimbarile foarte eficiente, deoarece trebuie doar schimbate sau adaugate filtre. Pe de alta parte, aceste filtre trebuie sa parseze datele, lucru care rezulta in overhead, asa ca nu putem spune ca Pipes&Filters este un stil arhitectural avantajos din punct de vedere al timpului.`
2. Folosirea constantelor globable incalca criteriul de modularitate al continuitatii.
`FALS. Folosirea constantelor globale nu poate incalca acest criteriu, ba chiar contribuie la el, deoarece atunci cand trebuie facuta o schimbare, aceasta este efectuata intr-un singur loc in cod. De exemplu modificarea dimensiunii unui array care apare in mai multe locuri in cod, schimbarea trebuie efectuata intr-un singur loc, nu peste tot unde e definit un array cu dimensiunea respectiva.`
3. Un sistem care respecta criteriul de modularitate al decompozabilitatii il va respecta aproape sigur si pe cel al compozabilitatii, deoarece acesta din urma de refera la posibilitatea compunerii unui sistem din module.
`FALS. Cele doua criterii prezinta niste diferente foarte relevante.Decompozabilitatea se refera la abilitatea de a imparti un sistem in mai multe module, fiecare cu o responsabilitate specifica. Ideea este ca asa fiecare modul poate fi inteles, dezvoltat si testat in izolare, chiar si de catre echipe diferite. Acest lucru ajuta la gestionarea complexitatii.Compozabilitatea, pe de alta parte, se refera la abilitatea de a asambla aceste componente/module mai mici in componente mai mari, si chiar`

`intr-un sistem mai mare, functional. Acest lucru implica faptul ca aceste module mici sunt proiectate astfel incat sa poata lucra impreuna, cu interfete clare. Compozabilitatea permite refolosirea componentelor si flexibilitate in design.Asadar, un sistem poate fi decompozabil, adica poate fi "spart" in mai multe parti componente, dar nu avem garantia ca aceste componente pot fi usor reasamblate sau folosite in orice alt context.`

4. Un avantaj principal al stilului arhitectural stratificat (Layered Architecture) este asigurarea criteriului de modularitate al protectiei.
   `FALS.Layered Architecture imparte aplicatia in "straturi logice" diferite. Fiecare strat are un rol specific si interactioneaza cu straturile care sunt imediat deasupra sau dedesubt. Acest fapt poate contribui la respectarea criteriului de modularitate al protectiei, dar nu o asigura automat. Criteriul se refera la ideea ca fiecare modul software ar trebui sa aiba cat mai putine privilegii posibile. Acesta este un principiu important de securitate, deoarece reduce suprafata de atac si limiteaza impactul potential al unui modul compromis. Asadar, stilul arhitectural stratificat nu asigura respectarea principiului de modularitate al protectiei, deoarece acest lucru depinde de modul in care straturile sunt implementate`
5. Un corp de mobila modular care vine impachetat pe bucati si care trebuie asamblat(gen IKEA) a fost proiectat urmarit in special criteriul de modularitate al compozabilitatii.
   `FALS. Un corp de mobila modular este proiectat in asa fel incat, asambland piesele/diferitele componente dupa un anumit plan/ iinstructiuni sa putem obtine acelasi produs final, lucru ce este urmarit de criteriul decompozabilitatii. De altfel, criteriul compozabilitatii se refera, mai degraba, la posibilitatea de a ajunge la output-uri diferite, pornind de la aceleasi piese(De ex, galetile de LEGO vechi, care nu veneau cu instructiuni si puteai sa construiesti orice voiai).`
6. Importanta criteriilor si regulilor de modularitate este cu atat mai mare cu cat anvergura sistemului software proiectat este mai mare, cu alte cuvinte: importanta modularizarii este proportionala cu dimensiunea sistemului software.
   `ADEVARAT. Intr-adevar, criteriile de modularitate ajuta mult in gestionarea complexitatii unui proiect software, care inherent este mai mare in cazul unui sistem de anvergura mai mare. Respectand criteriile, schimbarile pot fi efectuate mai eficient, nefiind necesara modificari majore in intreg sistemul pentru o singura modificare.` Pe de alta parte, poate fi adus si argumentul ca este importanta respectarea criteriilor de modularitate chiar si in cazul unui sistem software de anvergura mai mica, pentru a asigura scalabilitatea pe viitor. Implicit, proiectele software sunt supuse multor schimbari prin definitie si trebuie sa fie proiectate astfel incat sa suporte eficient scalarea.`
7. Intre specificatorul de acces `protected` si criteriul de modularitate al protectiei exista o legatura stransa.
   `FALS. Sunt doua lucruri diferite, specificatorul de acces "protected" permite ca atributele si metodele pe care este aplicat sa fie`

`vizibile/accesibile doar din clasa respectiva sau din clasele care o mostenesc, pe cand criteriul de modularitate al Protectiei se refera, mai degraba, la faptul ca daca un modul este afectat, atunci eroarea nu se va propaga in multe module. Totusi, specificatorul de acces "protected" poate facilita implementarea criteriului, impreuna cu "private" si "package".`

8. In stilul arhitectural Repository, diferitele componente care proceseaza date sunt total independente unele de altele, cu exceptia faptului ca folosesc acelasi model de date.
   `ADEVARAT. Stilul arhitectural "Repository" consta in faptul ca mai multe creiere interactioneaza cu datele din repo (deci folosesc acelasi model de date). Ele interactioneaza independent unii de ceilalti, deoarece producatorii si consumatorii de date sunt independenti (nu trebuie sa isi faca griji de responsabilitati auxiliare, cum ar fi backup-ul si securitatea).`
9. The modularity criterion named continuity states that we have to continously search for new requirements for a system due to a software evolution law.
   `FALS. Criteriul continuitatii se refera la faptul ca orice schimbare adusa unui sistem trebuie sa rezulte in doar cateva module, care nu afecteaza arhitectura intregului proiect. Mai mult, clientul ar trebui sa poata sa foloseasca sistemul fara a fi afectat de schimbarea detaliilor interne`
10. Arhitectura unui sistem software se refera la detaliile de proiectare, deci arhitectura nu are legatura directa cu cerintele sistemului.
    `FALS. Cerintele sistemului sunt cele care ne specifica ce trebuie sa poata face sistemul in sine, deci ele si arhitectura sistemului sunt inseparabile, deoarece multe decizii de design se iau luand in considerare cerintele. De exemplu, cu siguranta nu vom alege stilul arhitectural Pipes&Filters pentru o aplicatie interactiva, avand in vedere limitarile acestuia cu privire la timpul de raspuns, unidirectionalitate si lipsa starii.`
11. Un avantaj principal al Pipes&Filters este ca se asigura un factor comun intre diferitele filtre implicate.
    `FALS. Prin definitie, filtrele sunt complet independente si nu depind una de alta, nici prin functionalitate dar nici prin stare comuna. Ideea acestui stil in sine este compunerea comportamentului filtrelor individuale pentru a obtine sistemul dorit, iar filtrele pot fi reutilizate fara conflicte. Asadar, nu se asigura un factor comun intre diferitele filtre implicate. Acest argument ar putea fi adus doar pentru faptul ca este necesar un format de date comun, dar acesta e un dezavantaj pentru majoritatea sistemelor, nu un avantaj.`
12. Conceptele "ascunderea informatiei" si "abstractizare" sunt sinonime, intrucat se refera la aceeasi actiune.
    `FALS. Conceptul de ascunderea informatiei se refera la faptul ca se ascund detaliile interne, de ex daca avem o metoda disponibila clientului sorteaza(vector), acesta nu trb sa stie algoritmul din spate(el poate fi modificat oricand-> quicksort, mergesort etc). Conceptul de abstractizare se`

`refera la prezentarea caracteristicilor esentiale, adica ce este important sa stie restul sistemului(Conceptul de "abstractizare" se axeaza pe reducerea complexitatii prin concentrarea pe aspectele esentiale, ascunzand detalii irelevante in contextul respectiv.). Mai mult ele nu pot fi sinonime, deoarece primul este doar o modalitate sa ii dam enhance la al doilea.Astfel, chiar daca are loc o schimbare, componentele client ar trebui sa fie afectate minimal, respectand principiul continuitatii.`

13. 13. Precizati ce stil arhitectural s-ar asocia cel mai bine cu:
    14. O ceapa:
        - **Layered Architecture:** deoarece acesta are o structura stratificata, in mijloc aflandu-se nucleul, care este cea mai importanta parte.
    15. O linie de productie:
        - **Pipes&Filters:** Formatul de date este produsul in sine, care este procesat in mod repetat in cadrul filtelor, care reprezinta masinariile de pe linia de productie. Pipe-urile reprezinta benzile rulante care doar transporta produsul spre urmatorul filtru.
    16. IDE:
        - **Repository:** Un IDE contine componente diferite precum editorul, compilatorul, debugger, linter, care trebuie sa interactioneze in mod eficient si sa impartaseasca date precum codul sursa, rezultate de compilare, erori, etc..
          Totodata, o modificare in codul sursa trebuie sa fie vizibila in timp real pentru alte componente.
          De asemenea, se asigura o izolare/decuplare avansata. Componentele unui IDE au un rol specific si nu trebuie sa ia in considerare cum functioneaza celelalte componente. Cu un depozit central, componentele pot ramane izolate si implicit pot fi dezvoltate, testate si imbunatatite independent.
14. Daca intr-un sistem ar trebui favorizata performanta de timp, ati opta pentru stilul arhitectural Layered?
    `FALS. Deoarece in stilul Layered, fiecare strat comunica doar cu straturile adiacente, asa ca nu este disponibila o comunicare foarte rapida. Daca stratul cel mai din exterior trebuie sa comunice cu nucleul, informatia va trece de 2 ori prin toate straturile(odata pentru a trimite mesajul, si odata pentru a primi raspunsul).

    Dar daca ar trebui favorizata securitatea?
    - **DA:** Daca trebuie favorizata securitatea, Layered Architecture este un stil potrivit, deoarece chiar daca un strat este compromis, atacatorul nu are acces la intregul sistem. Suprafata de atac in sine este semnificativ redusa.
      Totodata, abordarea de Layered Security ne spune ca mai multe masuri de securitate, luate pe fiecare strat al sistemului, sunt mai puternice decat masurile luate la nivelul intregului sistem.`

Curs 6

1. Principalul dezavantaj al testarii Top Down este crearea de drivere si stub-uri.
   `FALS. In cazul acestei testari, cream doar stub-uri in locul low-level module care inca nu sunt implementate sau testate. Driverele sunt specifice pt Bottom Up, unde inlocuiesc modulele de mai sus de cele din cluster-ul curent/frunzele la care ne aflam.`
2. Ideea de baza in testarea White Box este aceea de a ne asigura ca cel putin privita individual, o functie nu are niciun bug in nicio instructiune, intrucat toate instructiunile sunt verificate cu cate cel putin un test.
   `Dupa cum ne spune Dijkstra, testarea ne poate arata doar prezenta erorilor, nu si absenta. Intr-adevar, testarea White Box are ca scop sa se asigure ca toate statement-urile si conditiile au fost executate cel putin odata. Functia, privita si testata individual, este posibil sa nu aiba niciun bug, dar asta poate fi si din cauza unor teste care nu trateaza cazurile corecte.`
3. Partitiile echivalente se folosesc la sisteme cu foarte multe functii lungi si complexe si reprezinta grupuri de functii care sunt asemanatoare din punct de vedere al testarii. Astfel, din fiecare grup/partitie se vor alege una sau mai multe functii pentru care sa se scrie suite de teste black box.
   `FALS`
   Partitiile echivalente reprezinta domenii de date(intrare sau iesire) in care programul aflat sub testate are un comportament echivalent. Asadar, pentru a scrie teste cat mai cuprinzatoare, trebuie sa alegem test cases din toate partitiile.
   De exemplu, daca testam o functie care are ca date de intrare un array de maxim 100 de elemente, vom testa cu:
   - un array gol
   - un array cu un element
   - un array cu 50 elemente
   - un array cu 99 elemente
   - un array cu 100 elemente
4. Daca dorim sa verificam cat mai timpuriu principalele puncte de control si decizie din sistem, vom alege testarea Bottom-Up.
   `FALS. Testarea de tip Bottom-Up incepe cu tesatea modulelor low-level/frunza (se utilizeaza cluster-e in acest tip de testare). Modulele mai inalte/neimplementate sunt inlocuite de drivere. Pentru a testa timpuriu principalele puncte de control si decizie vom alege, in schimb, testarea Top-Down, care inlocuieste modulele low-level cu stub-uri, dar testeaza modulele principale de cele mai multe ori(prin urmare si principalele puncte de control si decizie).`
5. Principalul motiv pentru care testarea de integrare de tip Big-Bang nu este recomandata este acela ca, spre deosebire de alte tehnici de testare de integrare, aceasta identifica mai putine defecte.
   `FALS. Testarea de integrare de tip "Big Bang" presupune scrierea in`

`intregime a codului, asamblarea tuturor componentelor sistemului si abia apoi se face testarea intregului sistem. Aceasta abordare nu e deloc recomandata, pentru ca izolarea erorilor este complicata, pot aparea erori care nu par a fi initial corelate si e dificil sa le rezolvi pentru ca te pierzi prin ele. Problema nu este ca identifica mai putine defecte, ci ca odata ce corectezi o eroare descoperita, apar altele.`

6. Testarea ciclurilor pentru o functie cu doua cicluri imbricate(nested) implica scrierea a doua cazuri de test: unul sa parcurca instructiunile din ciclul interior si unul sa le parcurga pe cele din ciclul exterior.
   `FALS. Testarea functiilor cu cicluri imbricate presupune parcurgerea acestora din interior spre exterior: Incepem cu ciclul interior si setam parametrii la minim din cel exterior si aplicam apoi cele mai simple reguli de testare pe ciclul interior, pastrand valorile minime la cel exterior. (skip la intregul loop; o singura trecere prin loop; doua treceri prin loop; N si (N - 1) treceri prin loop. Fixam mai apoi ciclul interior la o valoare tipica si ma concentrez asupra celui exterior aplicand aceleasi reguli de testare.`
7. Un program care nu are cicluri(for, while) are numarul ciclomatic=1.
   `FALS. Numarul ciclomatic este influentat de numarul deciziilor simple din cadrul unui program, nu de structurile repetitive in sine.Intr-adevar, for si while contin o clauza if care verifica o conditie, asadar exista o decizie simpla care influenteaza numarul ciclomatic.In general, numarul ciclomatic este egal cu numarul de decizii simple + 1.`
8. Valoarea complexitatii ciclomatice a unei functii nu este influentata de numarul de instructiuni de ciclare(for, while), deoarece singurele instructiuni care incrementeaza valoarea complexitatii ciclomatice sunt cele de decizie de tip if..else.
   `FALS. Si instructiunile de ciclare au de evaluat o conditie de oprire, altfel s-ar crea un loop infinit. Prin urmare, fiecare instructiune de ciclare va contribui la complexitatea ciclomatica, alaturi de cele de decizie if-else(de ex avem un for(i = 0; i < 10; i++), se va verifica i < 10 la fiecare bucla, DAR complexitatea ciclomatica va creste doar cu 1).`
9. La testarea Blackbox avem nevoie si de cod pentru a verifica daca datele de test acopera toate caile din program.
   `FALS Testarea Blackbox prin definitie nu necesita cunoasterea codului, deoarece acestea introduc un input si observa output-ul general de sistemul aflat sub testare. Acest tip de testare ia in considerare specificatiile datelor de input si output si pentru un coverage cat mai mare efectueaza testarea partitiilor.`
10. Testele Whitebox nu pot garanta ca toate erorile dintr-o functie vor fi gasite, dar daca sunt dublate de teste Blackbox, se poate garanta ca vor fi gasite toate erorile din acea functie.
    `FALS. Dupa cum ne spune si Dijkstra, nicio forma de testare nu poate arata absenta erorilor, ci doar prezenta lor. Erorile aparute depinde de modul in`

`care sunt scrise testele, asa ca daca un anumit test case nu este luat in considerare din diferite motive, acea eroare nu va aparea in teste, dar poate aparea in production.Evident, combinarea mai multor tehnici de testare asigura un procent mai mare de erori detectate, dar nicio combinatie nu ne poate garanta descoperirea tuturor erorilor dintr-un sistem software.`

11. Testarea de regresie este bazata pe ideea ca atunci cand se opereaza schimbari intr-un modul, imediat dupa modificare, eforturile de testare trebuie concentrate asupra acelui modul.
`FALS. Ideea de baza a testarii de regresie este reluarea unui subset de teste care au fost rulate anterior. Bineinteles ca aceste teste au legatura cu modulul modificat, dar nu testeaza exclusiv acel modul.` In testele de regresie se reiau 3 clase de teste:
`- Cateva teste care testeaza toate functiile software` - Teste aditionale pe functii care ar putea fi afectate de schimbarea efectuata
`- Teste focusate pe componentele software care au fost modificate in sine`
12. Daca avem la dispozitie codul, nu este suficient sa scriem teste Whitebox, chiar daca fiecare instructiune a fost exersata de catre un test.
`ADEVARAT. Chiar daca fiecare instructiune a fost executata macar de un test, este nevoie si de alte teste pentru a detecta si alte erori care pot aparea. De exemplu, este nevoie de testarea intr-un mod special al ciclurilor imbricate, pornind de la cel mai interior ciclu, si setand iteratorul celor externe pe cea mai mica valoarea. Apoi se merge "in sus", setand ciclurile interne pe un iterator tipic, iar cele externe tot pe cel mai mic iterator.De asemenea, este nevoie si de testare Blackbox, pentru a testa diferitele partitii in care pot fi cuprinse datele.`
13. Smoke Testing se refera la testarea detaliata a anumitor module, pentru a detecta erori subtile strecurate in implementare.
`FALS. Smoke Testing se refera la testarea intregului software, compilat, concentrate pe a detecta erori care pot incurca sistemul din a isi executa functia. Scopul principal este de a detecta erori "show stopper", care au cea mai mare sansa de a intarzia proiectul.Intreg produsul este Smoke Tested zilnic, oferind echipei o idee realista despre progresul Integration Testing.`
14. Metoda de testare de integrare Big-Bang este mai rapida decat testarea de integrare Top-Down sau Bottom-Up, pentru ca permite integrarea simultana a tuturor modulelor.
`FALS. Intr-adevar, testarea Big-Bang permite integrarea simultana a tuturor modulelor, dar acest lucru creaza mai multe dezavantaje decat avantaje, deoarece erorile detectate nu pot fi localizate clar. Totodata, in urma rezolvarii unei erori, de multe ori apar altele, ducand la o "bucla infinita" de rezolvare a unor erori si aparitia altora.`
15. For the basis/independent path testing approach, we can compute directly based on the total function code, the exact number of tests that have to be written.
`FALS. Nu putem stii exact cate teste avem de scris, dar putem determina un`

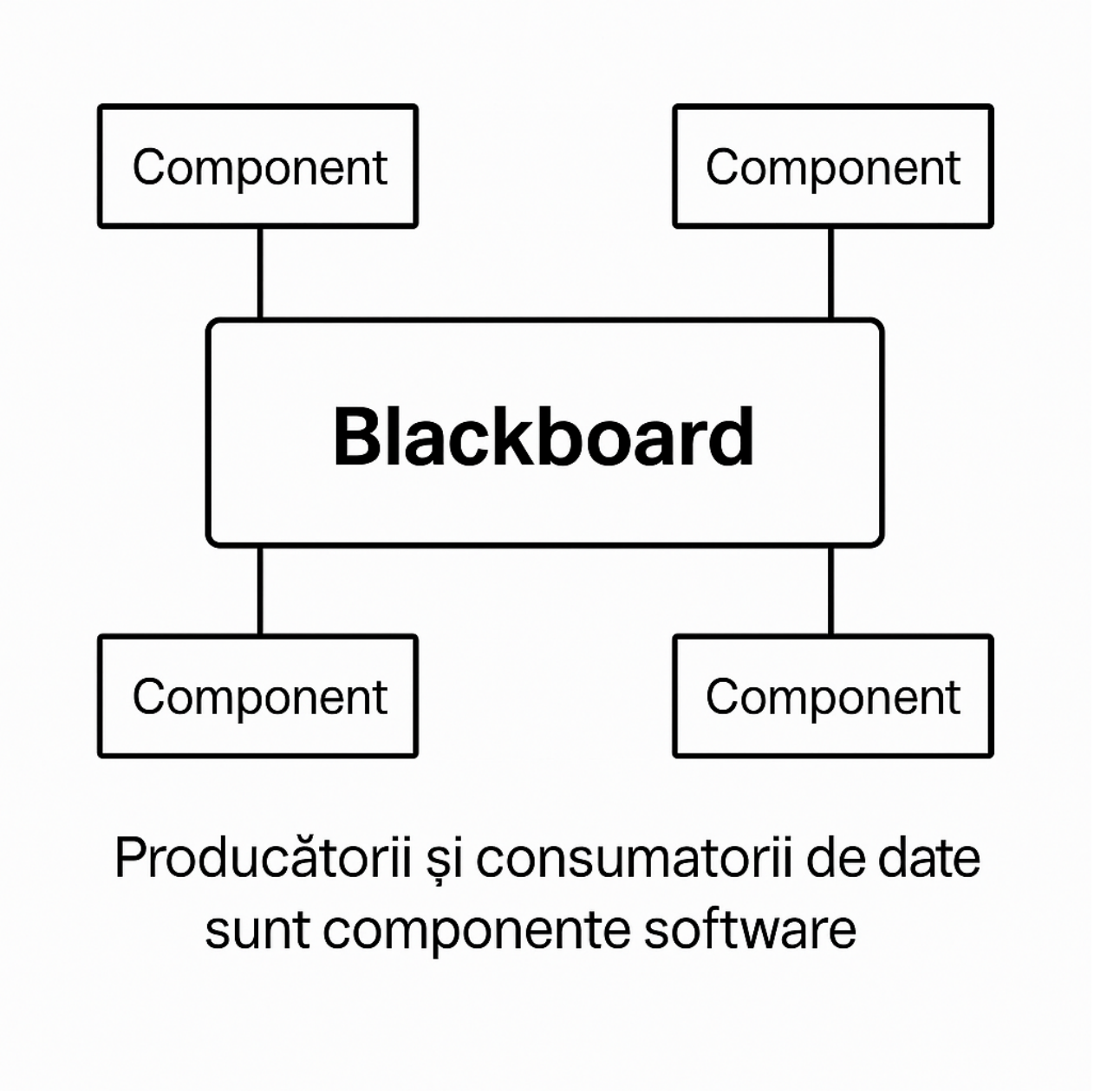
`numar minim de teste care trebuie efectuat, pe baza unui control flow graph, pe baza caruia observam complexitatea ciclomatica a functiei(nr de decizii simple + 1)`

16. When doing Blackbox testing on a function having as an input a list of elements, there are some special test cases that have to be designed for that.
    `ADEVARAT. In cazul unei liste de elemente(o colectie)testam : lista goala, lista cu un element, lista cu un nr diferit de elemente, accesul la primul, ultimul element si accesul la elementul din mijloc`

Varianta 2024

1. In tehnica CRC Cards, fiecare card CRC ar trebui sa reprezinte mai multe clase, mai exact toate clasele implicate intr-un anumit scenariu.
    `FALS. Prin utilizarea tehnicii de modelare CRC Cards(use case driven) noi vom identifica clasele impreuna cu responsabilitatile si colaboratorii acestora. In plus fiecare CRC card o sa reprezinte o singura astfel de clasa, impreuna cu responsabilitatile(metodele) si colaboratorii ei(alte clase cu care interactioneaza), prezentam implementarea solutiei, modul de utilizare si eficienta.`
2. Atunci cand refactorizam o functie, adica ii modificam modul de implementare fara a-i modifica functionalitatea, testele de tip black box vor trebui actualizate.
    `FALS. Testele de tip black box nu sunt teste scrise pe baza codului, ele sunt teste bazate pe input si output-uri asteptate. Prin urmare, daca functionalitatea nu i se modifica, ar trebui sa ne asteptam la acelasi output. Totodata, ar fi bine totusi sa ne mai scriem cateva teste noi, pentru ca, desi edge case-urile raman aceleasi, avem nevoie sa testam performanta noii implementari pentru a ne asigura ca functia a fost factorizata intr-un sens bun si are un timp mai bun de executie decat in vechea versiune.`
3. Costul adaugarii unei noi functionalitati in software este in general acelasi indiferent de momentul in care schimbarea apare pe parcursul de viata al software-ului.
    `FALS. Adaugarea unei noi functionalitati in software genereaza costuri diferite in functie de momentul in care se alege sa se implementeze raportat la viata SW-ului. Cel mai usor de integrat o schimbare(si cea mai putin costisitoare) este chiar in fata de inceput cand se dau cerintele initiale, apoi gradul odata cu inceperea dezvoltarii produsului, orice schimbare costa mai mult, pentru ca incetineste dezvoltarea produsului, iar dupa release, costurile sunt mult mai mari deoarece orice schimbare poate sa si deterioreze produsul.`
4. O linie ascendenta(crescatoare) intr-un Burndown Chart indica faptul ca task-urile au fost finalizate mai repede decat fusese planificat.
    `FALS. O linie intr-un Burndown chart indica in general progresul/regresul echipei intr-o iteratie. O linie ascendenta indica faptul ca task-urile nu`

`sunt finalizate corespunzator si tot se gasesc probleme/bug-uri care adauga la efortul ramas pentru sprint.`



5. La stilul arhitectural Blackboard/Repository producatorii si consumatorii de date sunt membrii echipei de dezvoltare care implementeaza sistemul.
   `FALS. În stilul arhitectural **Blackboard/Repository**, producătorii și consumatorii de date nu sunt membrii echipei de dezvoltare, ci sunt componente software din sistemul implementat. **Blackboard architecture**: există o componentă comună de tip "tablă" (blackboard) unde diverse componente specializate (producători și consumatori) scriu și citesc date în mod indirect prin intermediul acesteia.- Membrii echipei de dezvoltare **proiectează și implementează** aceste componente, dar în cadrul sistemului, producătorii și consumatorii sunt modulele software care interacționează cu blackboard-ul, nu oamenii care le-au creat.
6. Atat relatia de agregare cat si cea de compozitie implica faptul ca, daca clasa de baza este stearsa, toate clasele derivate de asemenea vor fi sterse.